Continual Dyna With Model Aging

Abstract

Dyna-style planning is attractive for a continual agent because it reuses a learned model to improve value estimates without requiring more real interaction. In a nonstationary world, the same mechanism can amplify obsolete knowledge. This proposal studies model freshness as a first-class Core RL question: can a small online agent age or distrust model entries so that planning remains useful after the environment changes?

Standalone Study Summary

This study asks when a continual Dyna agent should trust its learned model after the world changes. The RL problem is a continuing gridworld whose layout changes midstream; the agent keeps acting and learning without reset. The implemented methods compare no planning, random keep-model planning, oracle model flushing, recency-weighted model aging, and recency/error-gated model aging. The extended experiment varies planning budgets 0, 1, 5, 20, model-handling rules, and aging half-lives 250, 750, 1500, 4000; the main metrics are average reward, stale-backup rate, model error, planning TD magnitude, and post-change recovery window. The current result shows that freshness-aware sampling sharply reduces stale backups. Reward improvement is real in some high-budget settings but depends on the half-life and budget, so the final claim should be about search-control freshness rather than universal reward superiority.

Proposal Template Answers

Focused RL question: In a continual Dyna agent, when should learned model entries stop receiving planning computation after the environment changes? The proposal studies planning trust and search-control freshness, not simply whether "more planning" improves reward.

Setting and testbed: The main testbed is a continuing gridworld with a midstream layout change and no agent reset. It is large enough for planning to matter and small enough to label stale model entries, which makes model-freshness diagnostics possible. A second stochastic or gradual-drift environment is planned to test whether the finding survives outside an abrupt maze change.

Implemented comparison: The implemented comparison includes no planning, keep-model Dyna, oracle model flushing, recency aging, and recency/error gating across planning budgets and aging half-lives. The oracle flush condition is explicitly diagnostic; it is not a realistic algorithm.

Observation or figure that answers the question: The main evidence is the relationship among late reward, stale-backup rate, model error, planning budget, and half-life. A useful result can be a tradeoff curve rather than one winner: the question is how model freshness changes the value of computation.

Compute need and fallback: The 20-seed extended grid is complete. If no additional environment is run, the honest fallback is to submit this as an abrupt-change model-freshness study and explicitly reserve gradual/stochastic drift for future work.

Independent Research Scope

This integrated proposal is an independent planning study. It should not be collapsed into the Dyna Planning Budget proposal. The budget proposal asks how much planning helps or hurts around a change; this report asks how a continual agent should decide which parts of a learned model deserve planning after knowledge becomes stale. The core object is search control under model aging.

The report does not study replay buffers or offline model learning. The model is compact, updated online, and queried for planning backups. This distinction is central to the course constraints: old experience is not stored and replayed; instead, the agent maintains a learned model whose entries may become obsolete.

Evidence Level

Evidence level: strong integrated-candidate evidence for abrupt nonstationarity and model-freshness diagnostics. The completed result uses 20 seeds, four planning budgets, multiple model-handling rules, and four aging half-lives. It directly measures stale-backup rates and model error, so the evidence supports a mechanism claim rather than only a reward claim.

The evidence is not yet broad enough for a general nonstationary planning claim. The environment change is abrupt and deterministic, and stale entries are relatively easy to define. A fuller study should add gradual drift, repeated changes, and a stochastic transition environment. Until then, the conclusion should emphasize "freshness-aware search control reduces stale backups in an abrupt changing gridworld" rather than "model aging solves continual planning."

Research Motivation

The Alberta Plan gives learned models and planning a central role in a long-lived agent. The hard version of planning is not stationary gridworld acceleration; it is deciding which learned predictions still deserve computation. A replay-buffer framing would store old experience and sample it later, but this project instead uses a compact learned model whose entries are updated online. That distinction matters: the question is not how to train from old data, but how a streaming agent should allocate background computation when its model may be wrong.

The current Dyna Planning Budget study already shows a tradeoff: planning helps before a change, but keeping the old model produces high stale-backup rates afterward. The integrated proposal turns that diagnostic into a stronger research problem by testing model aging, recency weighting, and search-control rules.

Research Question

How should a small continual Dyna agent allocate planning backups when model knowledge has unknown freshness?

Subquestions:

- - How much pre-change benefit is lost when model entries are aged aggressively?
- - Can stale-backup diagnostics predict post-change recovery?
- - Is flushing the model too crude compared with gradual aging?
- - Does prioritizing recent model error improve real-step reward or only make planning look cleaner?

Related Work

The Alberta Plan frames transition models and background planning as components of a base agent. Average-reward learning/planning work motivates continuing

objectives. Classic Dyna and prioritized sweeping motivate model-based backups and search control. Continual RL foundations motivate process metrics such as recovery time and adaptation rather than final policy score alone.

Useful local sources:

• - resources/alberta_plan_related/alberta_plan_2208.11173.pdf
• - resources/alberta_plan_related/average_reward_learning_planning_2006.16318.pdf
• - resources/alberta_plan_related/rethinking_foundations_continual_rl_2504.08161.pdf

Research Method

The agent maintains a compact state-action model:

• - estimated next state,
• - estimated reward,
• - last update time,
• - count or confidence,
• - recent prediction error.

Planning variants:

• - No planning.
• - Random Dyna with kept model.
• - Flush-on-change oracle diagnostic.
• - Recency-aged Dyna: planning priority decays with time since real observation.
• - Error-gated Dyna: entries with recent high model error are temporarily suppressed.

A stale-error-prioritized variant, where backups are chosen by a mixture of TD priority and model freshness, is a planned extension rather than part of the current result. The completed extended evidence uses keep_model, no_planning, oracle_flush, recency_aging, and recency_error_gate.

The flush-on-change variant is not realistic; it is an upper-bound diagnostic showing what would happen if the agent had a perfect change detector.

Experimental Design

Primary environment:

• - Continuing gridworld with goals and hazards.
• - Midstream layout change with no reset.
• - Planning budgets 0, 1, 5, 20.
• - Aging half-lives 250, 750, 1500, 4000 steps in the extended code path.
• - Seeds 0-19 for the current extended result.

Secondary environment:

• - Planned, not yet implemented: a small stochastic queue or random-walk model where transition probabilities drift gradually instead of changing abruptly. This prevents the result from depending only on a single blocked-maze event.

Metrics:

• - Real-step average reward.
• - Pre-change AUC.

• - Post-change recovery time.
• - Stale-backup rate.
• - Model one-step prediction error.
• - Planning utility: improvement in TD target or value estimate per backup.
• - Fraction of planning spent on entries not observed recently.

Experiment Design Rationale

The experiment uses a changing gridworld because stale model entries can be identified and counted. That diagnostic visibility is necessary for the research question: a reward curve alone cannot tell whether planning helps because the model is useful or hurts because the model is obsolete. Planning budgets 0, 1, 5, 20 separate the no-planning baseline, low-compute regime, and high-compute regime where stale backups can dominate.

The half-life sweep is the main scientific control. A very short half-life should distrust old knowledge quickly and may throw away still-useful structure; a long half-life should preserve more knowledge but risk stale backups. The correct outcome is therefore not necessarily one best half-life, but a map of when freshness helps and when it merely reduces planning.

Expected Results And Failure Modes

Expected pattern: random keep-model Dyna should achieve strong pre-change performance but high stale-backup rates after change. Flush should remove stale backups but may lose useful knowledge. Aging should form an intermediate regime: less stale than keep-model and less abrupt than flush, with performance depending on the half-life.

Failure modes:

• - Aging may be equivalent to lowering planning budget.
• - Model error may be delayed because the agent stops visiting changed states.
• - A deterministic gridworld may make stale detection too easy.
• - Stochastic drift may make aggressive aging unnecessarily pessimistic.

Interpretation Standard

The proposal succeeds if it explains when planning computation should be trusted. It does not need a universally best aging rate; a tradeoff curve is scientifically valuable if it reveals how model freshness, planning budget, and recovery interact.

Results

The current extended result is:

experiments/alberta_core_rl/results/continual_dyna_model_aging/20260709T024602Z_extended

Main result:

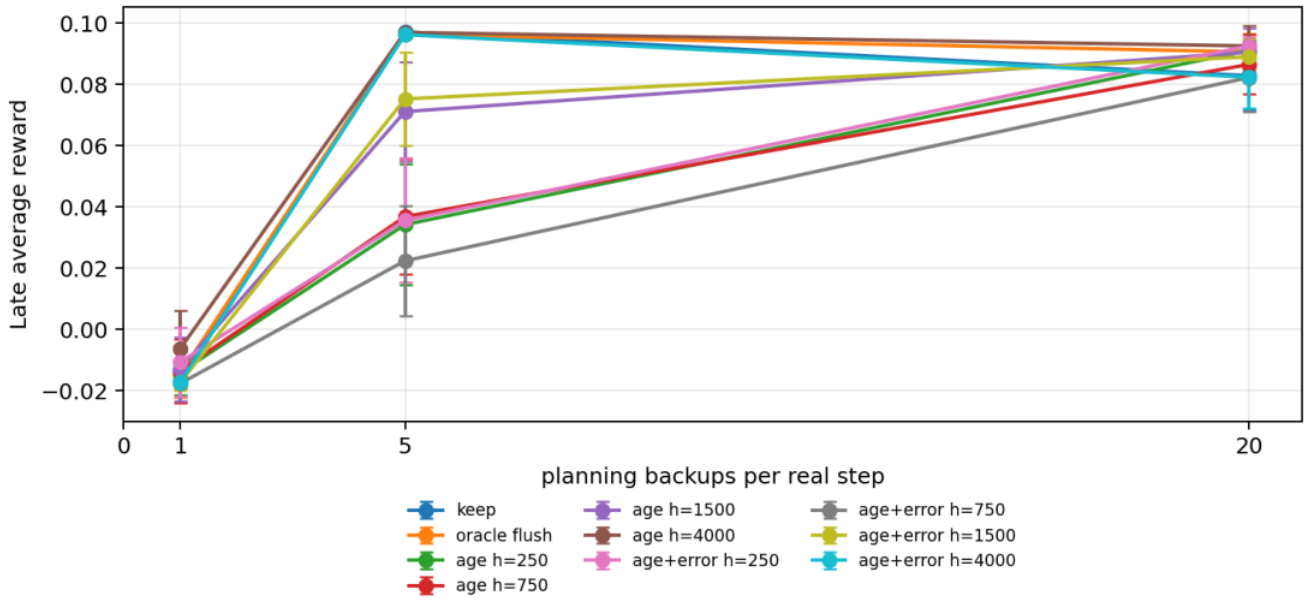
• - Freshness-aware sampling reduces stale-backup rate without oracle change detection.
• - At planning budget 20, keep-model stale-backup rate in the late post-change window is 0.213 +/- 0.065. Recency aging reduces this to essentially zero at half-life 250, to 0.00058 +/- 0.00017 at half-life 1500, and to 0.0204 +/- 0.0044 at half-life 4000.

• - Late reward at planning budget 20 is 0.0828 +/- 0.0113 for keep-model and 0.0906 +/- 0.0060 for oracle flush. Recency aging ranges from 0.0865 to 0.0925 depending on half-life; recency/error gating ranges from 0.0822 to 0.0925.

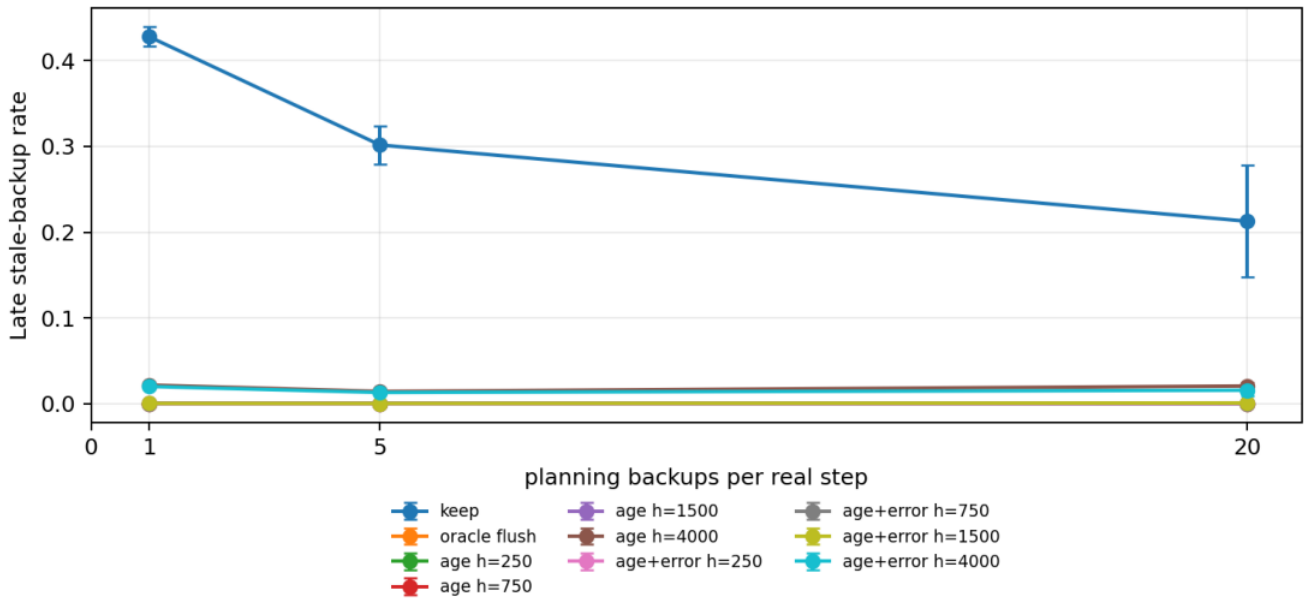
• - At planning budget 5, keep-model and oracle flush have strong late reward around 0.096-0.097, while aggressive aging half-lives 250 and 750 reduce stale backups but also hurt late reward. This shows that freshness control is not automatically better; it must match the planning budget and environmental time scale.

• - At planning budget 1, stale-backup reduction is visible but reward remains weak for all methods. There may not be enough planning computation for search-control improvements to translate into behavior.

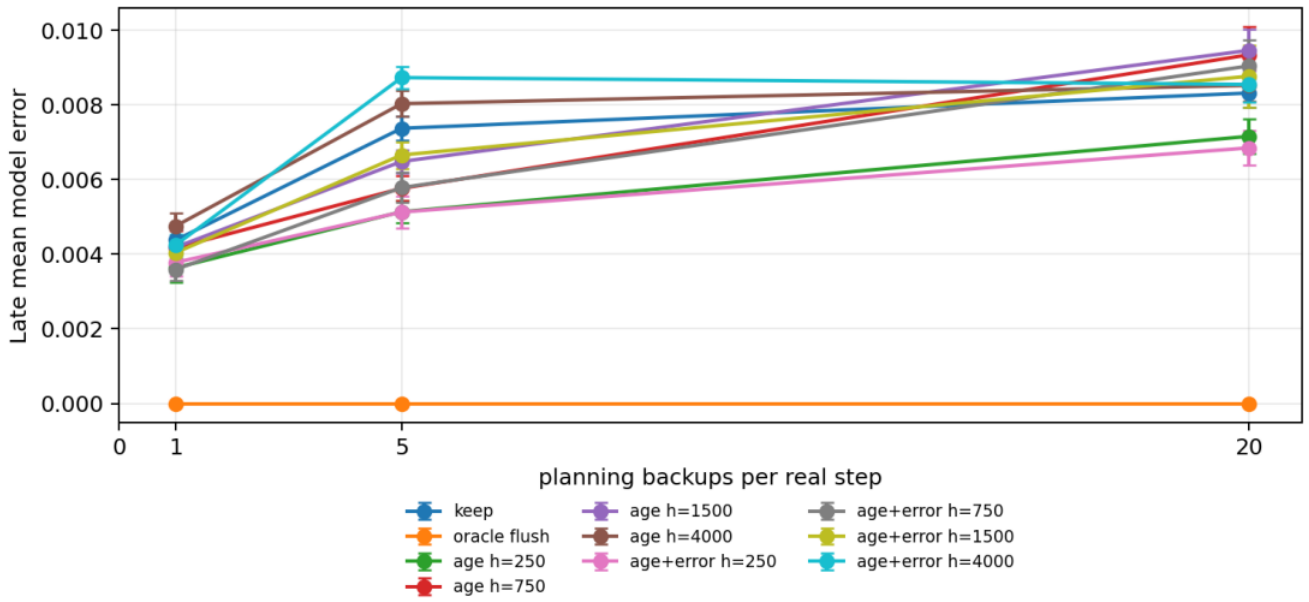
Main figures below use late post-change seed-tail condition summaries with 95% confidence intervals. For recency-based methods, the figure legend explicitly separates half-lives 250, 750, 1500, and 4000; this prevents the plot from hiding sensitivity to the aging time scale. This is more readable than the earlier many-series learning curves and focuses the plot on the research question: whether fresh model selection reduces stale planning after change.



Late average reward by planning budget, model mode, and half-life.



Late stale-backup rate by planning budget, model mode, and half-life.



Late model error by planning budget, model mode, and half-life.

Reviewer Critique And Revisions

Strict reviewer challenge: "This is just Dyna-Q with a changing maze." Response: the proposal must report model-freshness diagnostics and planning utility, not only reward.

Strict reviewer challenge: "Flush-on-change is unrealistic." Response: it is retained only as an oracle diagnostic; realistic variants must use recency or prediction error computed from the stream.

Per-proposal audit matrix:

```
| Reviewer angle | Critique | Action taken | Remaining risk |
|---|---|---|---|
| Alberta Plan | Planning should be about learned models in ordinary experience, not offline replay. | Uses an online learned model with planning backups and no replay buffer. | The environment is still compact and synthetic. |
| Planning reviewer | Reward alone cannot diagnose stale planning. | Reports stale-backup rate, model error, planning budget, and half-life. | Planning utility per backup needs a fuller table. |
| Nonstationarity reviewer | Abrupt change may make aging look too easy. | Conclusion is limited to abrupt changing gridworlds. | Needs gradual/stochastic drift and repeated changes. |
| Statistics | Half-life sensitivity can be hidden by one curve. | Current report figures separate half-life in the legend and report numerical examples. | A compact Pareto table would improve readability. |
| Strict instructor | Do not claim universal reward superiority. | Report emphasizes stale-backup reduction and tradeoffs. | Some reward comparisons remain budget-dependent. |
```

Threats To Validity

The current environment uses an abrupt gridworld layout change, which makes stale model entries easy to define and inspect. That is useful for mechanism diagnosis but may overstate how cleanly freshness can be detected in stochastic or gradually drifting worlds. The extended run now sweeps half-life, and the results show sensitivity: aggressive aging can reduce stale backups while hurting reward at planning budget 5. Conclusions should therefore emphasize the search-control tradeoff, stale-backup rate, model error, and recovery windows rather than only mean reward. A fuller study should add a drifting queue or stochastic transition stream to test whether the same planning-control pattern survives outside a maze.

Conclusion

This proposal turns Dyna from a generic "more planning helps" story into a sharper Core-RL question about when a continual agent should trust its learned model. The extended run supports the idea that model freshness is a first-class planning variable: recency and recency/error gating sharply reduce stale backups without oracle change detection. The reward story is more conditional than the pilot suggested; the best freshness setting depends on planning budget and half-life. The next step is a second nonstationary environment with gradual or stochastic drift.

Reproduction

Current extended result:

```
cd /mnt/shared-storage-user/yupeng/Core-RL

PYTHONNOUSERSITE=1 MPLCONFIGDIR=/mnt/shared-storage-user/yupeng/Core-RL/.mplconfig \
  /data/yupeng/conda_envs/core-rl/bin/python experiments/alberta_core_rl/scripts/run_experiment.py \
  --config experiments/alberta_core_rl/configs/continual_dyna_model_aging/config_extended.json

PYTHONNOUSERSITE=1 MPLCONFIGDIR=/mnt/shared-storage-user/yupeng/Core-RL/.mplconfig \
  /data/yupeng/conda_envs/core-rl/bin/python experiments/alberta_core_rl/scripts/plot_report_figures.py \
  --result-dir experiments/alberta_core_rl/results/continual_dyna_model_aging/20260709T024602Z_extended \
  --kind dyna-aging
```